Machine Learning

Lecture 4

Today

- Ensembling methods combine multiple models and can perform better than the individual members.
  - We've seen many individual models (KNN, decision trees)
- Bagging: Train models independently on random "resamples" of the training data.

Bias/Variance

Bias/variance

bias: how wrong the expected prediction is (corresponds to under-fitting)

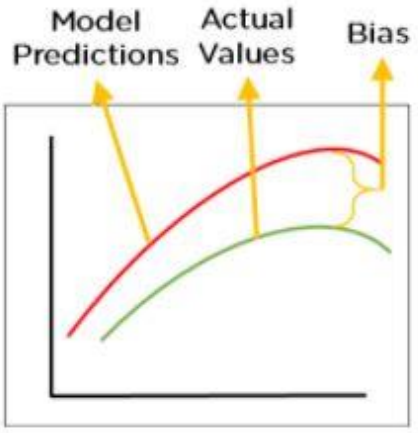
variance: the amount of variability in the predictions (corresponds to over-fitting)

Variance is the very opposite of Bias. During training, it allows our model to 'see' the data a certain number of times to find patterns in it. If it does not work on the data for long enough, it will not find patterns and bias occurs. On the other hand, if our model is allowed to view the data too many times, it will learn very well for only that data. It will capture most patterns in the data, but it will also learn from the unnecessary data present, or from the noise.

Bias

Bias is simply defined as the inability of the model because of some difference or error occurring between the model's predicted value and the actual value.

Bias is one type of error that occurs due to wrong assumptions about data such as assuming data is linear when in reality, data follows a complex function.



Bias

- The high-bias model will not be able to capture the dataset trend.
- It is considered as the underfitting model which has a high error rate.
- It is due to a very simplified algorithm.
- For example, a linear regression model may have a high bias if the data has a non-linear relationship.

Ways to reduce high bias in Machine Learning

- **Use a more complex model:** by increasing the number of hidden layers in the case of a deep neural network. Or we can use a more complex model like Polynomial regression for non-linear datasets.

- **Increase the number of features:** By adding more features to train the dataset will increase the complexity of the model. And improve its ability to capture the underlying patterns in the data.

- **Increase the size of the training data:** Increasing the size of the training data can help to reduce bias by providing the model with more examples to learn from the dataset.

variance

variance is the variability of the model that how much it is sensitive to another subset of the training dataset

We can define variance as the model's sensitivity to changes in the data. Our model may learn from noise. This will cause our model to consider trivial features as important.

variance

Variance errors are either low or high-variance errors.

- **Low variance:** means that the model is less sensitive to changes in the training data and can produce consistent estimates of the target function.
- **High variance:** means that the model is very sensitive to changes in the training data and can result in significant changes in the estimate of the target function when trained on different subsets of data from the same distribution.
- This is the case of overfitting when the model performs well on the training data but poorly on new, unseen test data.
- It fits the training data too closely that it fails on the new training dataset.

Ways to Reduce the reduce Variance:

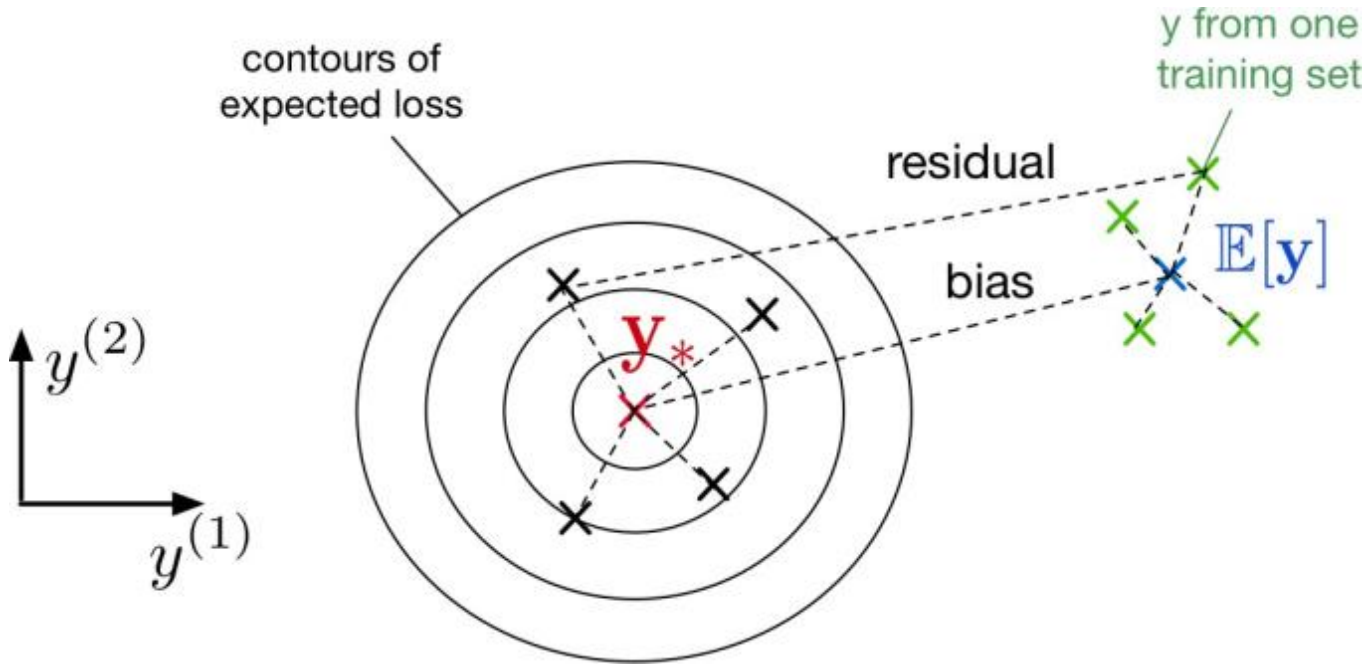
- **Cross-validation:** By splitting the data into training and testing sets multiple times
- **Feature selection:** By choosing the only relevant feature will decrease the model's complexity.
- **Ensemble methods:** It will combine multiple models to improve generalization performance. Bagging, boosting.
- **Simplifying the model:** Reducing the complexity of the model, such as decreasing the number of parameters or layers in a neural network.

In a similar way, Bias and Variance help us in parameter tuning and deciding better-fitted models among several built.

Overly Simple Model

An overly **simple** model (e.g. KNN with large k) might have

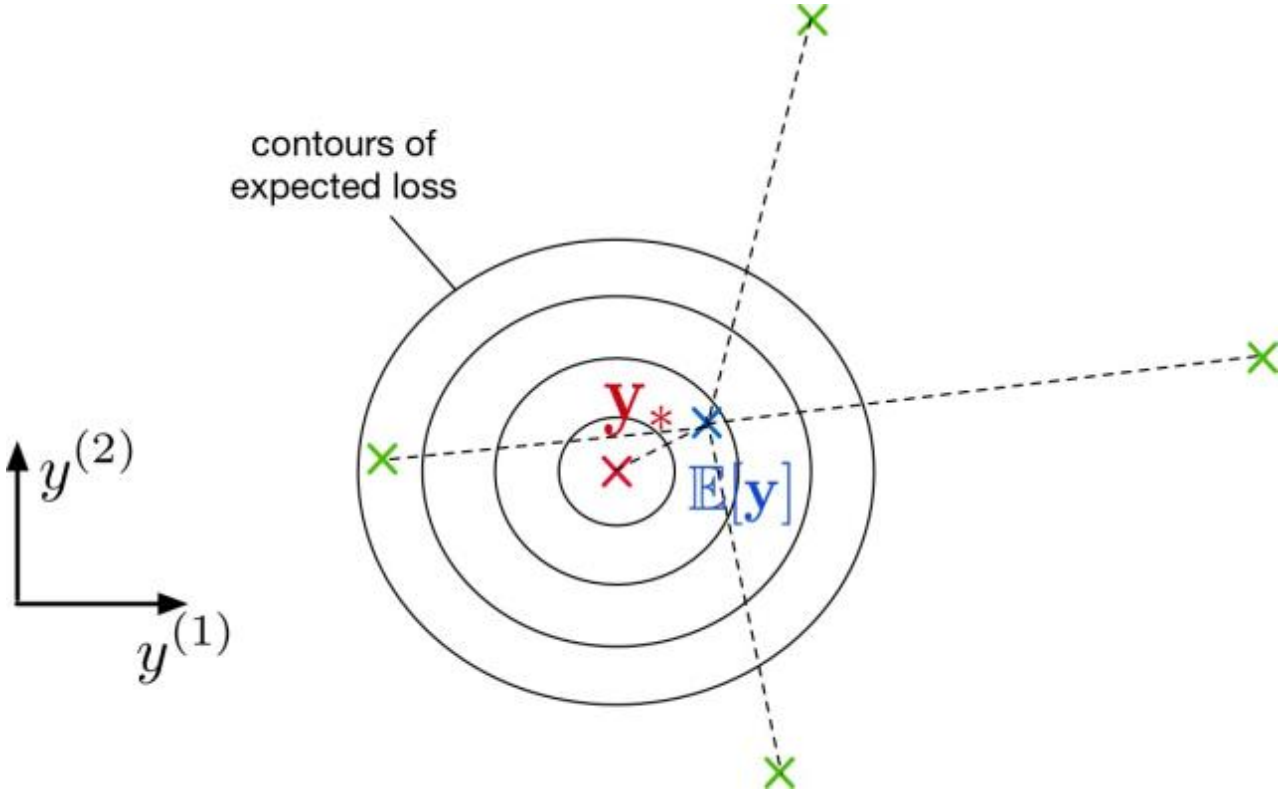
- **high bias**
  (cannot capture the structure in the data)
- **low variance**
  (enough data to get stable estimates)



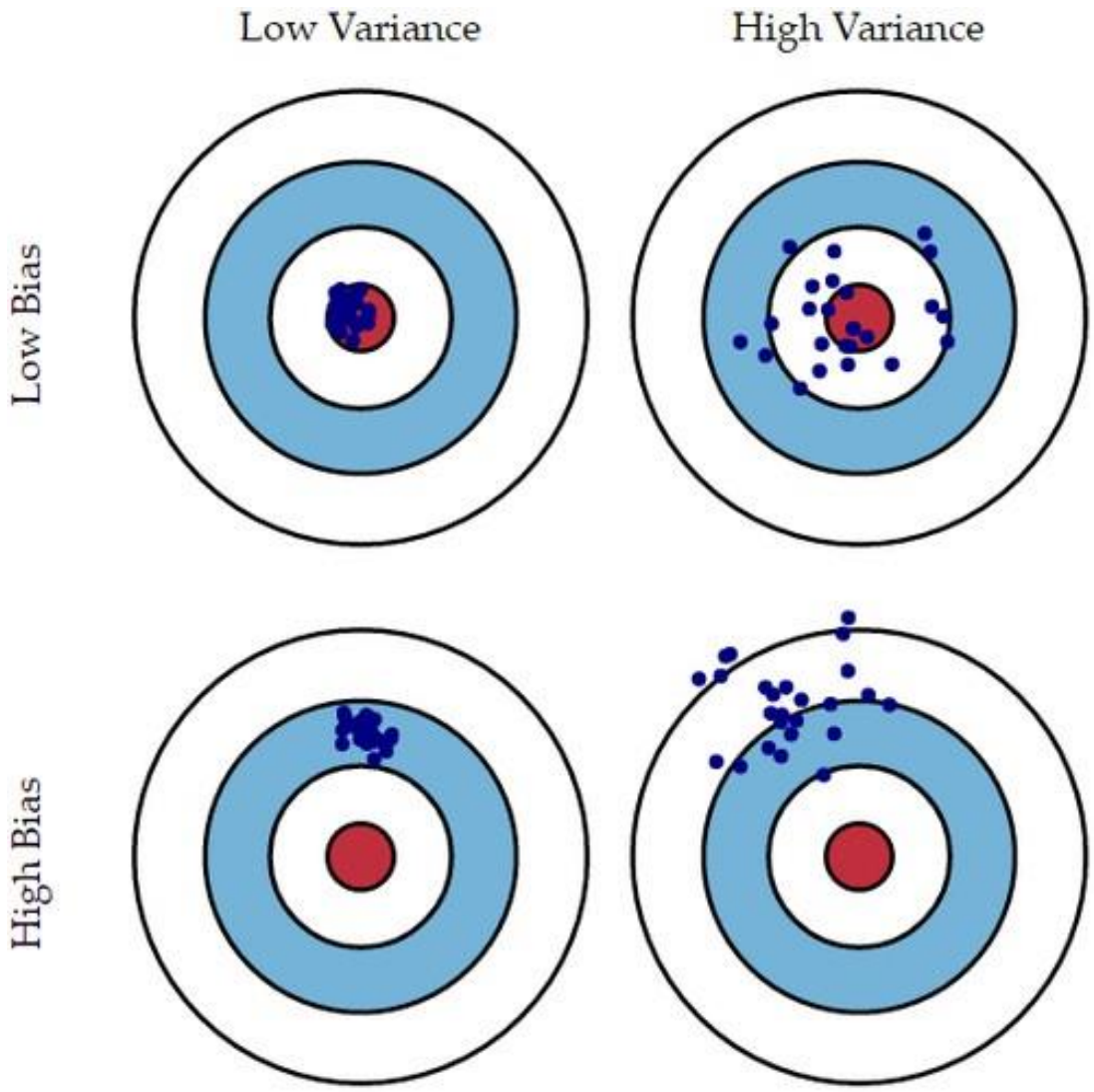
Overly Complex Model

An overly **complex** model (e.g. KNN with k = 1) might have

- **low bias**
  (learns all the relevant structure)
- **high variance**
  (fits the quirks of the data you happened to sample)



Visual of Bias/Variance Decomposition



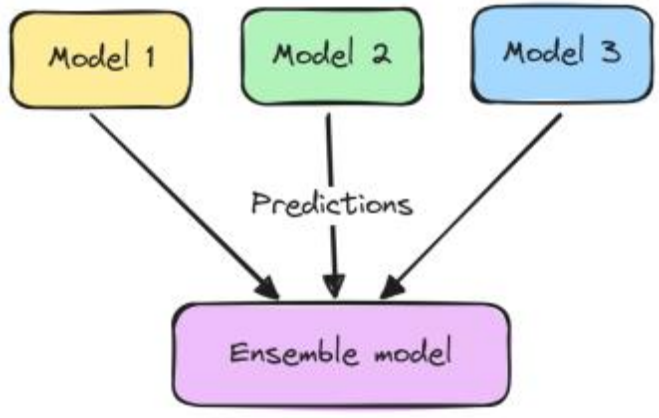
Bagging

Ensemble methods in machine learning

Ensemble methods are powerful techniques in machine learning that combine multiple models to improve overall prediction accuracy and model stability.

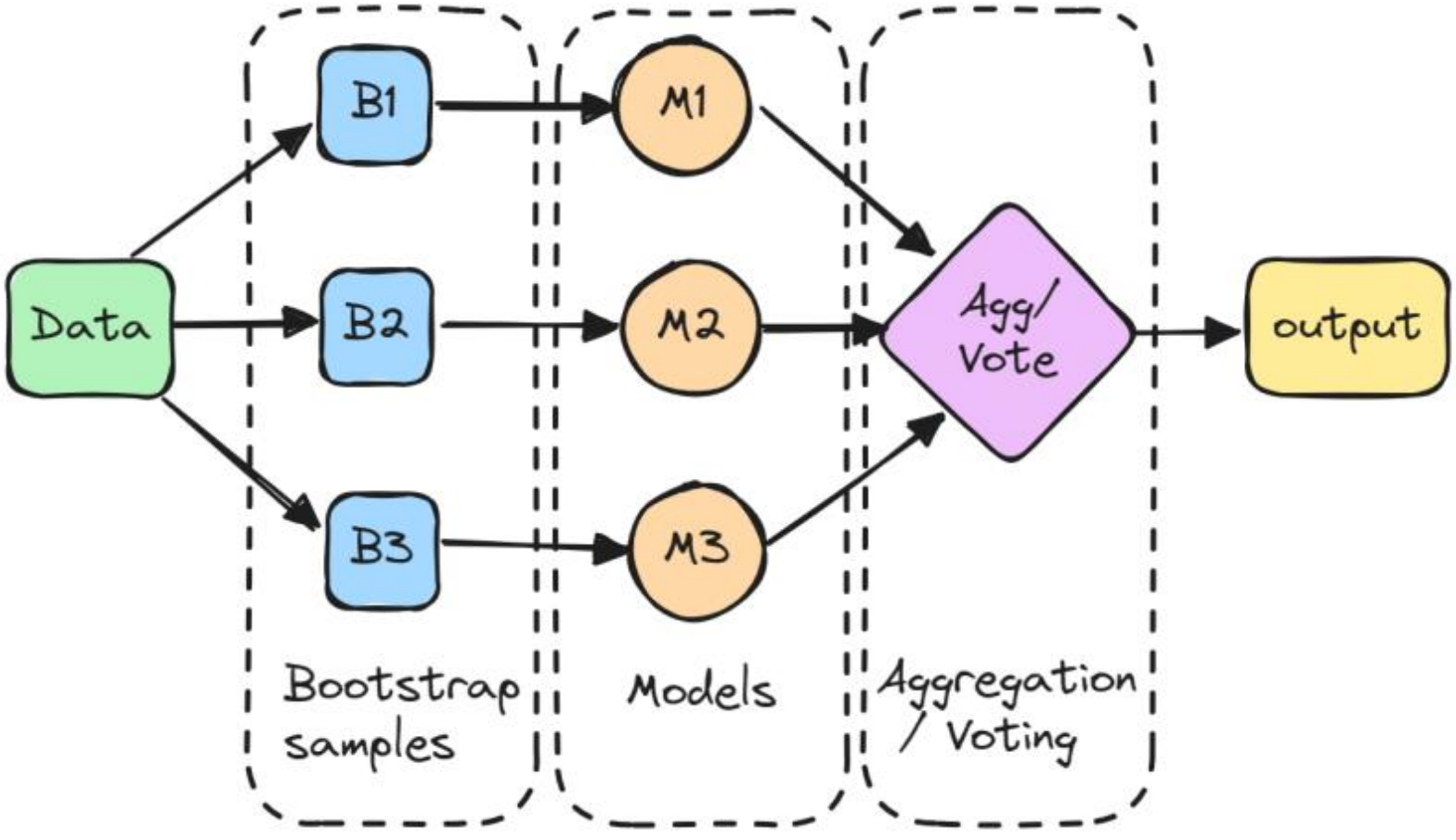
The basic idea is that a group of weak learners can come together to form one strong learner.

An ensemble model typically consists of two steps:
1.Multiple machine learning models are trained independently.
2.Their predictions are aggregated in some way, such as by voting, averaging, or weighting.



Bagging: The Idea

Bagging (bootstrap aggregating) is an ensemble method that involves training multiple models independently on random subsets of the data, and aggregating their predictions through voting or averaging.



Bagging

- Each model is trained on a random subset of the data sampled with replacement.
- The individual data points can be chosen more than once.
- This random subset is known as a bootstrap sample.
- By training models on different bootstraps, bagging reduces the variance of the individual models. It also avoids overfitting by exposing the constituent models to different parts of the dataset.
- The predictions from all the sampled models are then combined through a simple averaging to make the overall prediction.
- This way, the aggregated model incorporates the strengths of the individual ones and cancels out their errors.

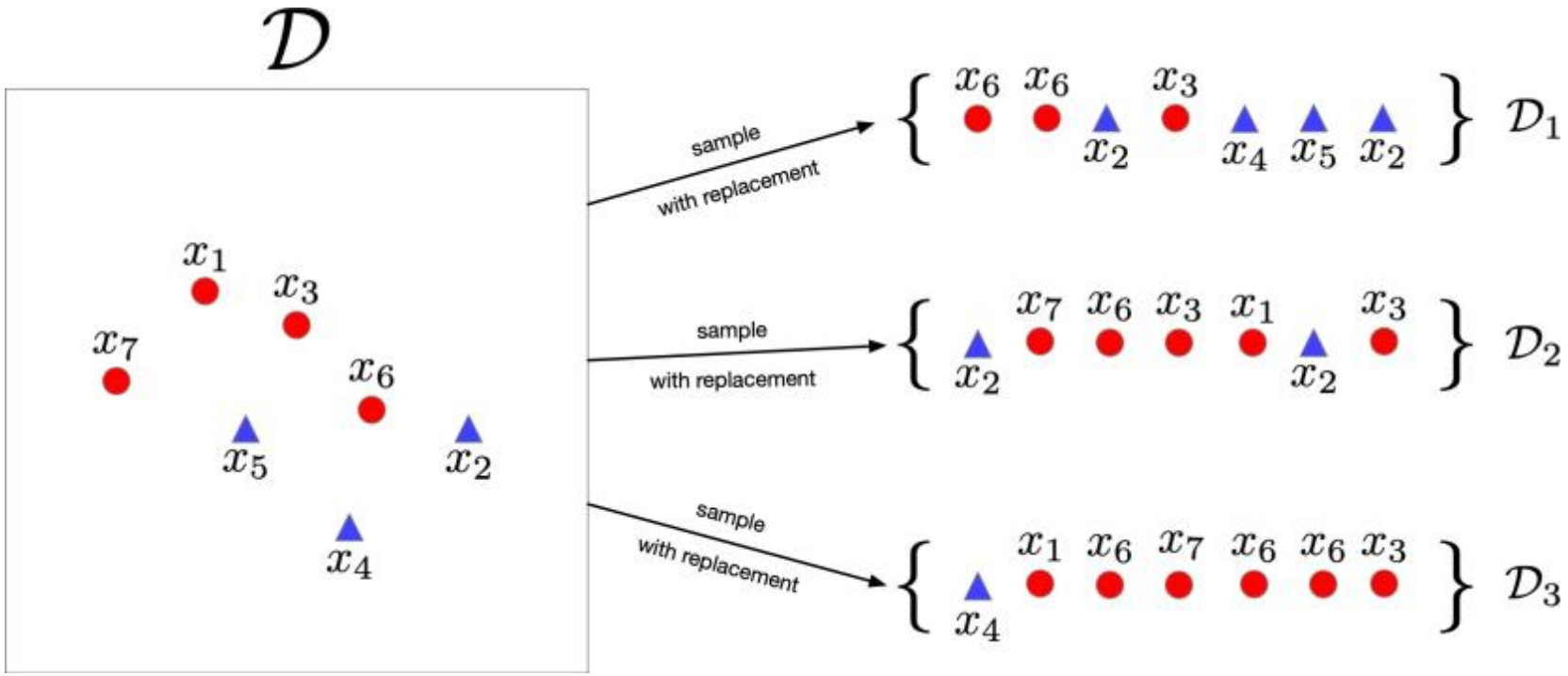
Bagging

- How does this affect the two terms of the expected loss?
  - **Bias:** unchanged,
    since the averaged prediction has the same expectation
  - **Variance:** reduced,
    since we are averaging over independent predictions

Bagging, also known as bootstrap aggregation, is the ensemble learning method that is commonly used to reduce variance within a noisy dataset.

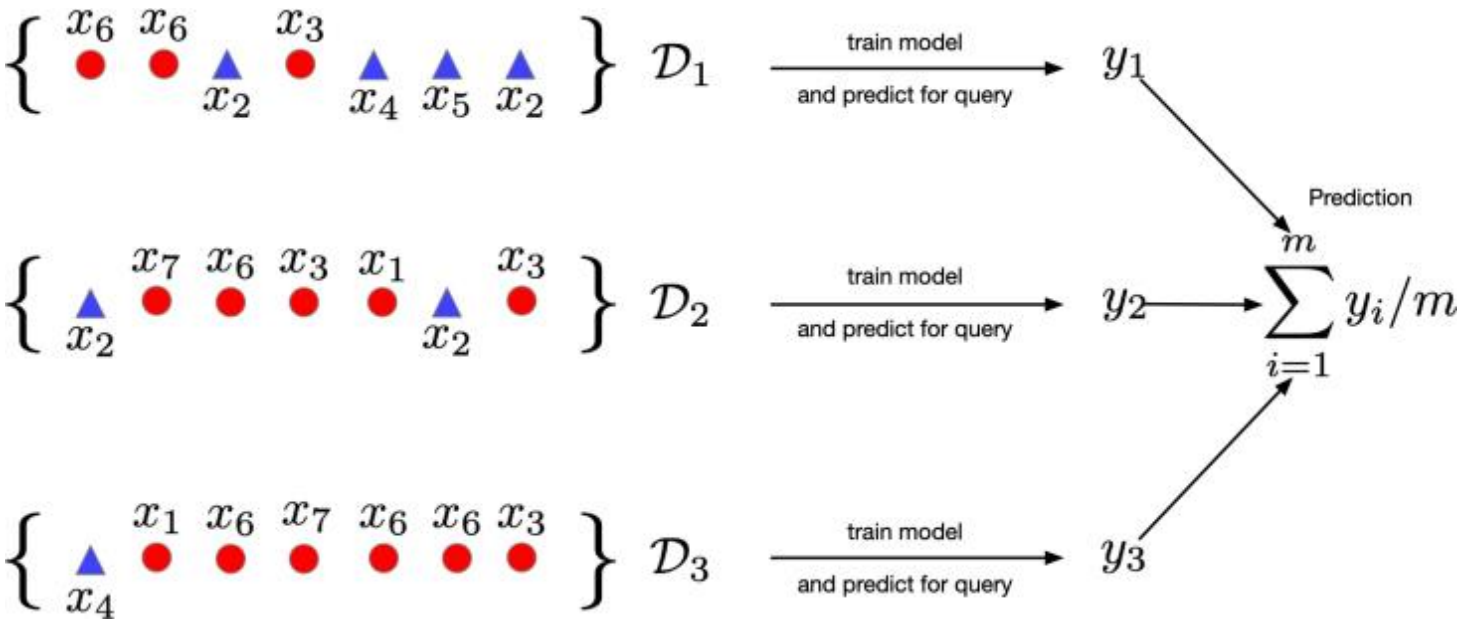
Bagging Example 1/2

Create $m = 3$ datasets by sampling from D with replacement. Each dataset contains $n = 7$ examples.



Bagging Example 2/2

Generate prediction y_i using dataset D_i.
Average the predictions.



Bagging Properties

- A bagged classifier can be stronger than the average model.
  - E.g. on "Who Wants to be a Millionaire", "Ask the Audience" is much more effective than "Phone a Friend".
- But, if m datasets are NOT independent, don't get the $\frac{1}{m}$ variance reduction.
- Reduce correlation between datasets by introducing *additional* variability
  - Invest in a diversified portfolio, not just one stock.
  - Average over multiple algorithms, or multiple configurations of the same algorithm.

Bagging Summary

Reduces over-fitting by averaging predictions. In

most competition winners.

A small ensemble often better than a single great model.

Limitations:

- Does not reduce bias in case of squared error.
- Weighting members equally may not be the best.
  Weighted ensembling often leads to better results if members are very different.

Random Forests

Instead of building a single decision tree and use it to make predictions, build many slightly different trees
Combine their predictions using majority voting

The main two concepts behind random forests are:

The wisdom of the crowd — a large group of experts are collectively smarter than individual experts
Diversification — a set of uncorrelated tress

A supervised machine learning algorithm

Classification (predicts a discrete-valued output, i.e. a class)
Regression (predicts a continuous-valued output) tasks.

Random Forests

- A trick to reduce correlation between bagged decision trees:
  For each node, choose a random subset of features
  and consider splits on these features only.
- Probably the best black-box machine learning algorithm.
  - works well with no tuning.
  - widely used in Kaggle competitions.

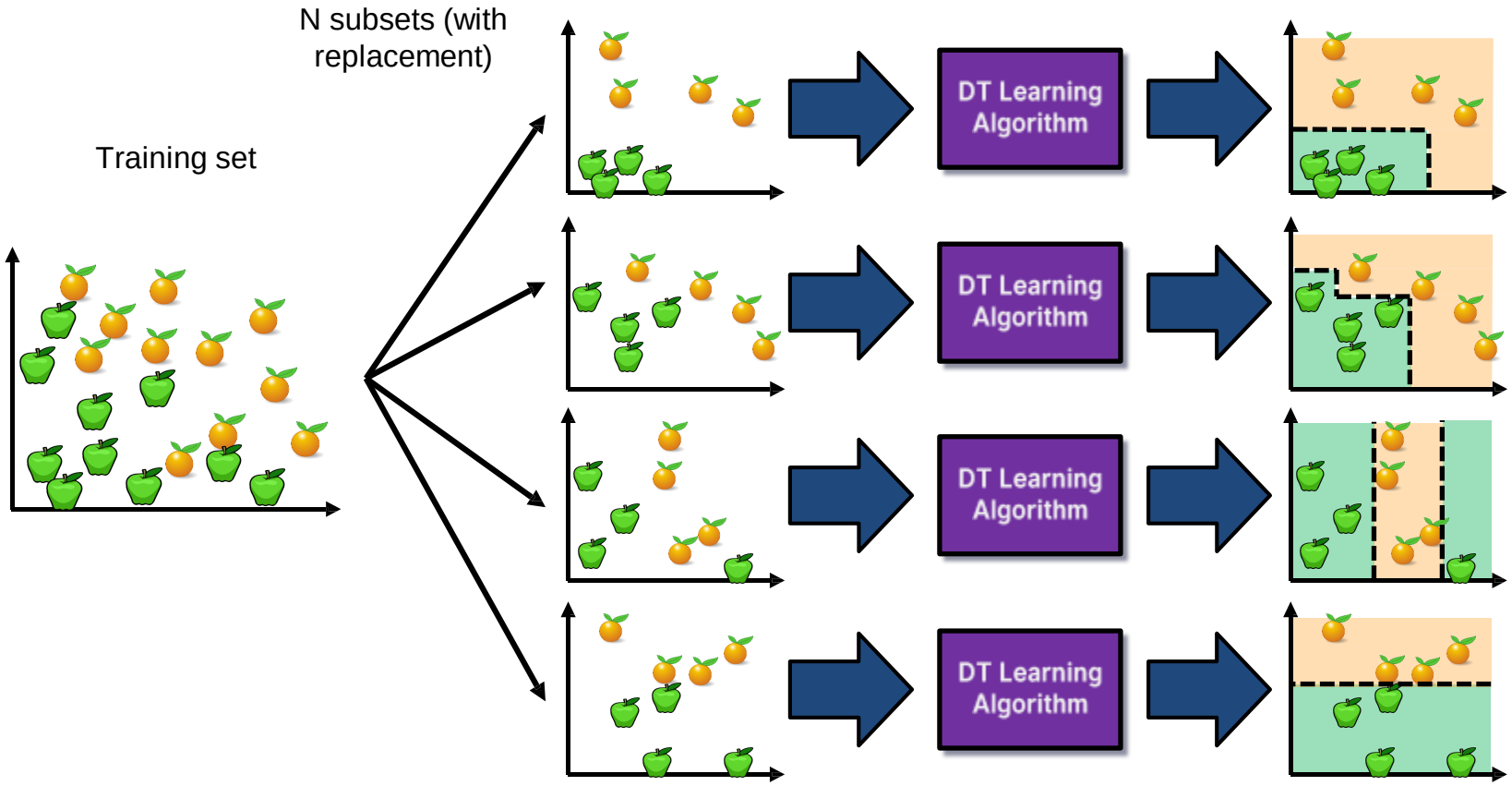
Random Forest

We have a single data set, so how do we obtain slightly different trees?

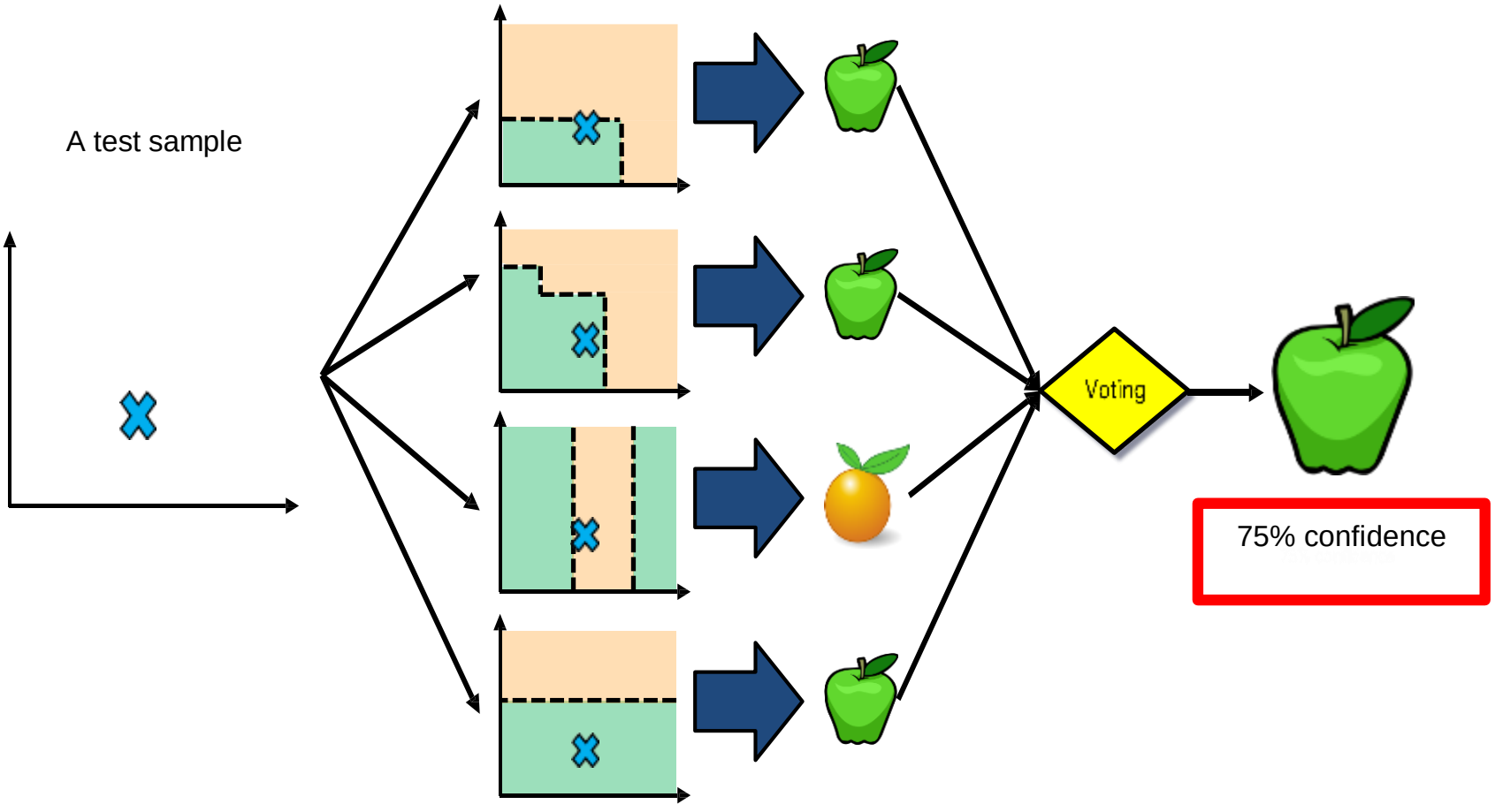
1. Bagging (Bootstrap Aggregating):
    Take random subsets of data points from the training set to create N smaller data sets
    Fit a decision tree on each subset
    Results in low variance.

2. Random Subspace Method (also known as Feature Bagging):
    Fit N different decision trees by constraining each one to operate on a random subset of features

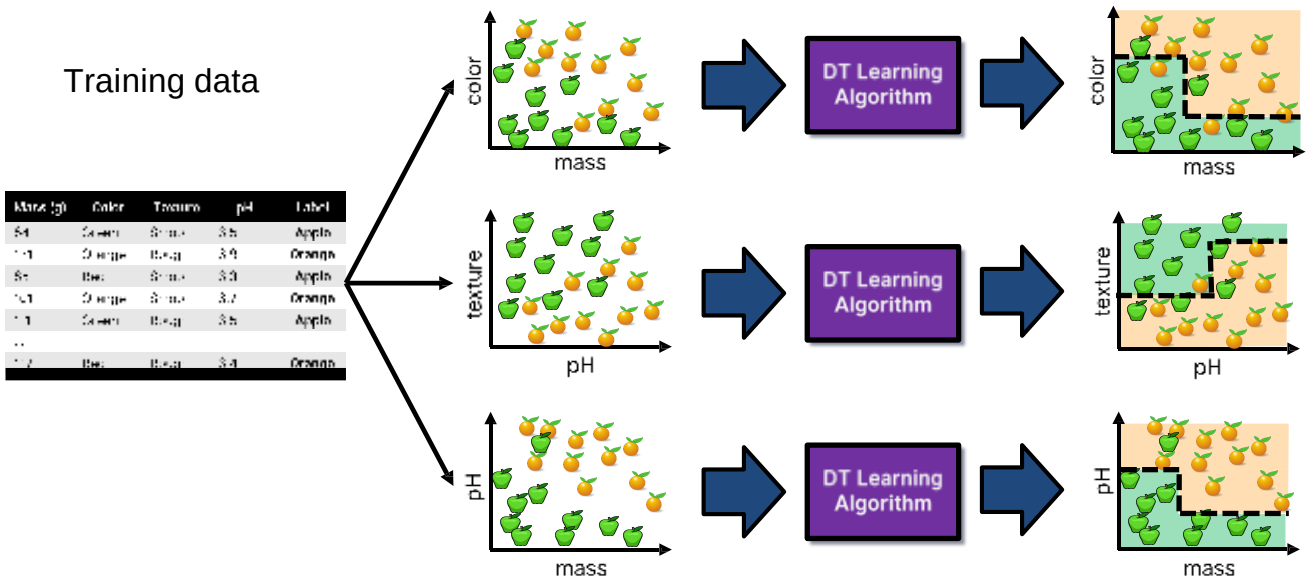
Bagging at training time



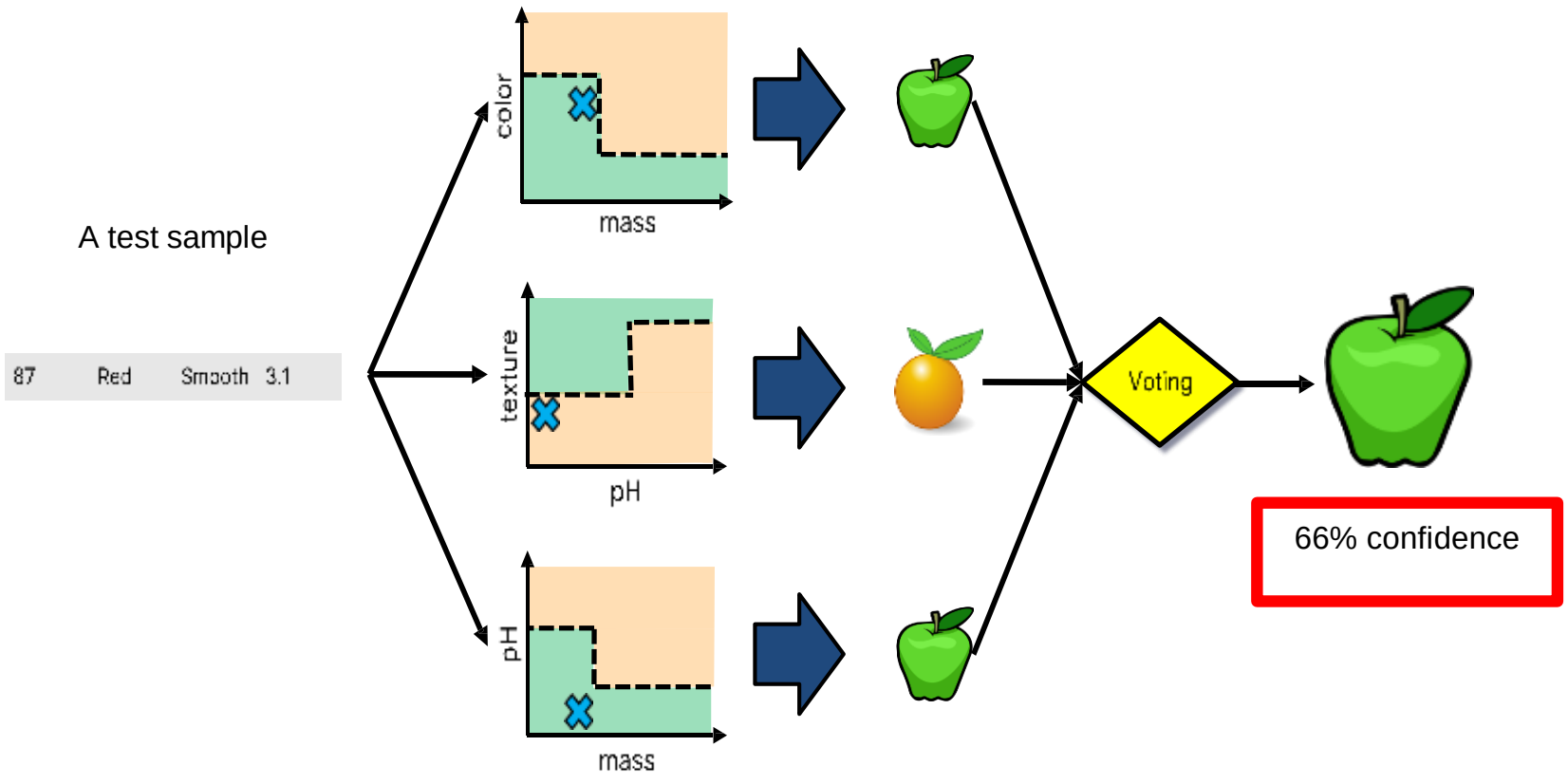
Bagging at inference time



Random Subspace Method at training time



Random Subspace Method at inference time



Optimization

Solving the Minimization Problem

Goal is to minimize the cost function J ($\mathbf{w}$).

Recall: the minimum of a smooth function (if it exists) occurs at a critical point, i.e. point where the derivative is zero.

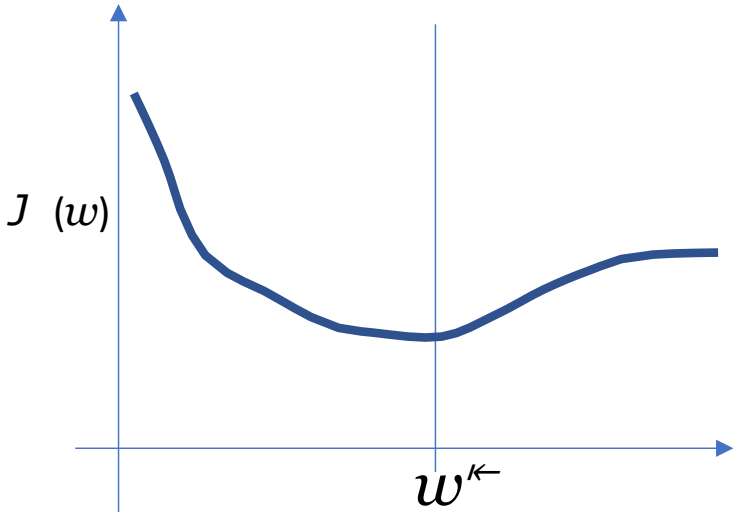
$$\nabla_{\mathbf{w}} \mathcal{J} = \frac{\partial \mathcal{J}}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial \mathcal{J}}{\partial w_1} \\ \vdots \\ \frac{\partial \mathcal{J}}{\partial w_D} \end{pmatrix}$$

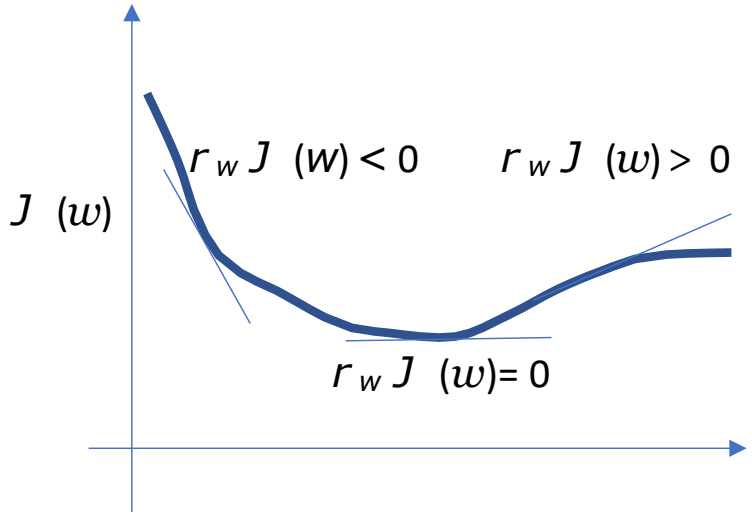
Solutions may be direct or iterative.

- Direct solution: set the gradient to zero and solve in closed form — directly find provably optimal parameters.
- Iterative solution: repeatedly apply an update rule that gradually takes us closer to the solution.

Minimizing 1D Function

- Consider $J(w)$ where w is 1D.
- Seek $w = w^*$ to minimize $J(w)$.
- The gradients can tell us where the maxima and minima of functions lie
- **Strategy:** Write down an algebraic expression for $\nabla_w J(w)$. Set $\nabla_w J(w) = 0$. Solve for w.





Iterative Solution: Gradient Descent

- Many optimization problems don't have a direct solution.
- A more broadly applicable strategy is gradient descent.
- Gradient descent is an iterative algorithm, which means we apply an update repeatedly until some criterion is met.
- We initialize the weights to something reasonable (e.g. all zeros) and repeatedly adjust them in the direction of steepest descent.

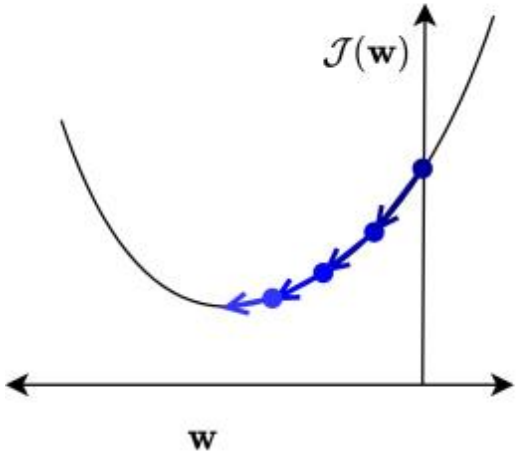
Deriving Update Rule

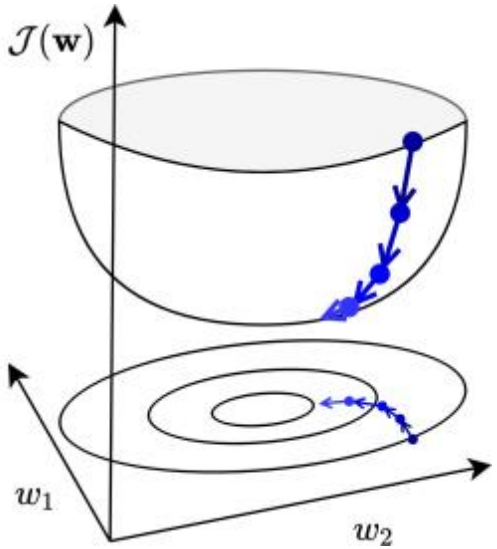
Observe:

- if $\partial J / \partial w_j > 0$, then decreasing J requires decreasing w_j. if
- $\partial J / \partial w_j < 0$, then decreasing J requires increasing w_j.

The following update always decreases the cost function for small enough α (unless $\partial J / \partial w_j = 0$):

$$w_j \rightarrow w_j - \alpha \frac{\partial J}{\partial w_j}$$





Setting Learning Rate

Gradient descent update rule:

$$w_j \rightarrow w_j - \alpha \frac{\partial J}{\partial w_j}$$

$\alpha > 0$ is a learning rate (or step size).

- The larger α is, the faster **w** changes.
- Values are typically small, e.g. 0.01 or 0.0001.
- We'll see later how to tune the learning rate.

Gradient Descent Intuition

- Gradient descent gets its name from the gradient, the direction of fastest *increase*.

$$\nabla_{\mathbf{w}} \mathcal{J} = \frac{\partial \mathcal{J}}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial \mathcal{J}}{\partial w_1} \\ \vdots \\ \frac{\partial \mathcal{J}}{\partial w_D} \end{pmatrix}$$

- Update rule in vector form:

$$\mathbf{w} \rightarrow \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}}$$

Update rule for linear regression:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)}) \, \mathbf{x}^{(i)}$$

- Gradient descent updates **w** in the direction of fastest *decrease*.
- Once it converges, we get a critical point, i.e. $\frac{\partial \mathcal{J}}{\partial \mathbf{w}} = \mathbf{0}$.

Why Use Gradient Descent?

- Applicable to a much broader set of models.
- Easier to implement than direct solutions.
- More efficient than direct solution for regression in high-dimensional space.
  - The linear regression direction solution $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{t}$ requires matrix inversion, which is $O(D^3)$.
  - Gradient descent update costs $O(ND)$ or less with stochastic gradient descent.

Feature Mappings

Feature Mapping

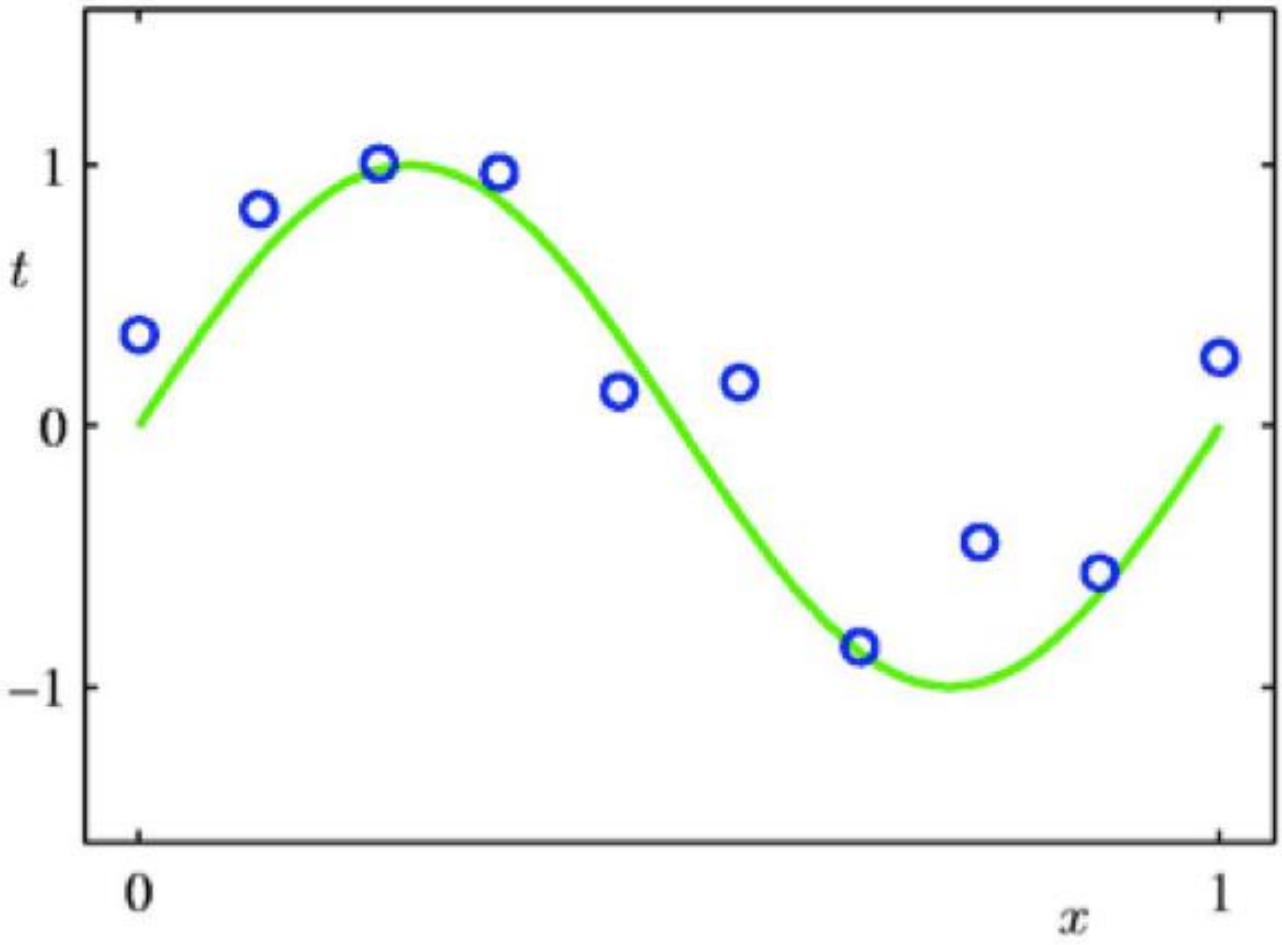
Can we use linear regression to model a non-linear relationship?

Map the input features to another space $\boldsymbol{\psi}(\mathbf{x}) : \mathrm{R}^D \rightarrow \mathrm{R}^d$.

Treat the mapped feature (in R^d) as the input of a linear regression procedure.

Feature mapping is a technique used in data analysis and machine learning to transform input data from a lower-dimensional space to a higher-dimensional space, where it can be more easily analyzed or classified.

Modeling a Non-Linear Relationship

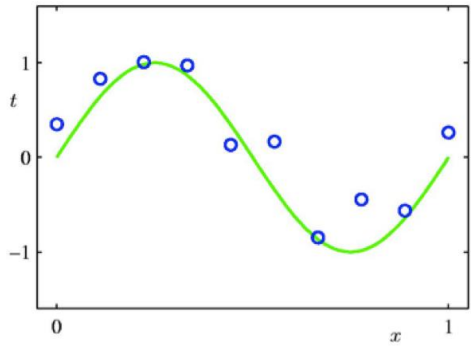


Polynomial Feature Mapping

Fit the data using a degree-M polynomial function of the form:

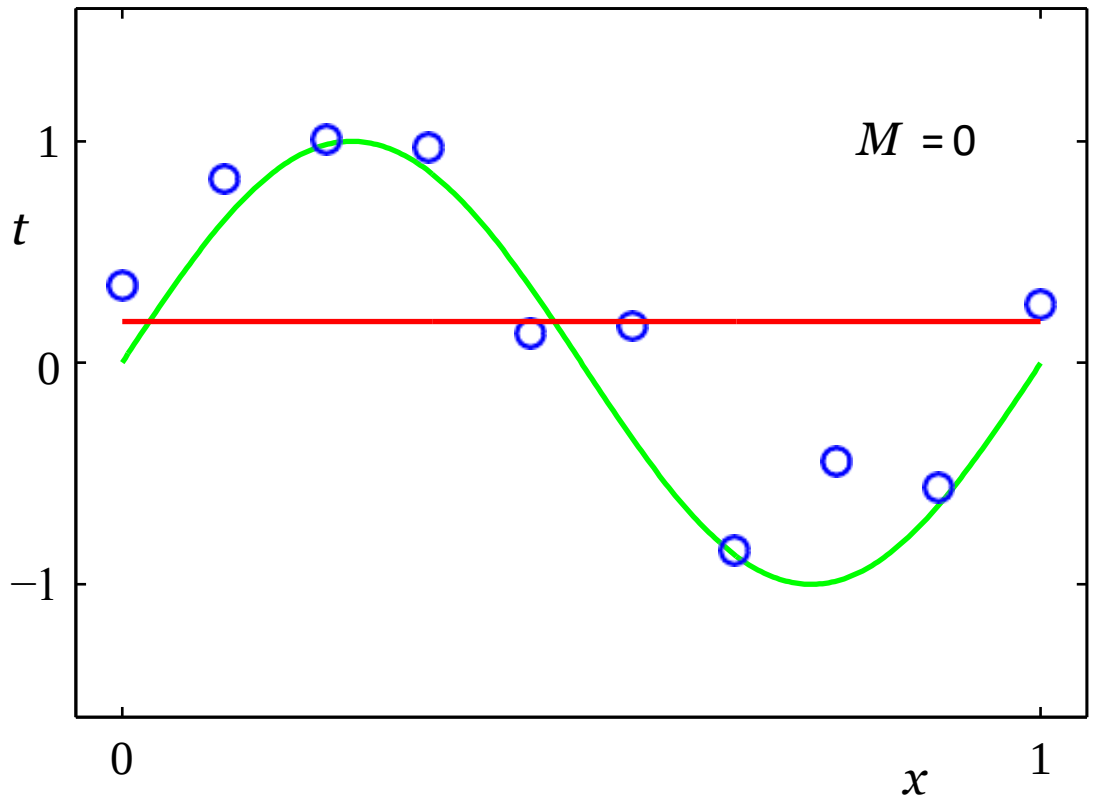
$$y = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{i=0}^{M} w_i x^i$$

- The feature mapping is $\boldsymbol{\psi}(x) = [1, x, x^2, \ldots, x^M]^T$.
- $y = \boldsymbol{\psi}(x)^T \mathbf{w}$ is linear in $w_0, w_1, \ldots$. Use
- linear regression to find **w**.



Polynomial Feature Mapping with $M = 0$

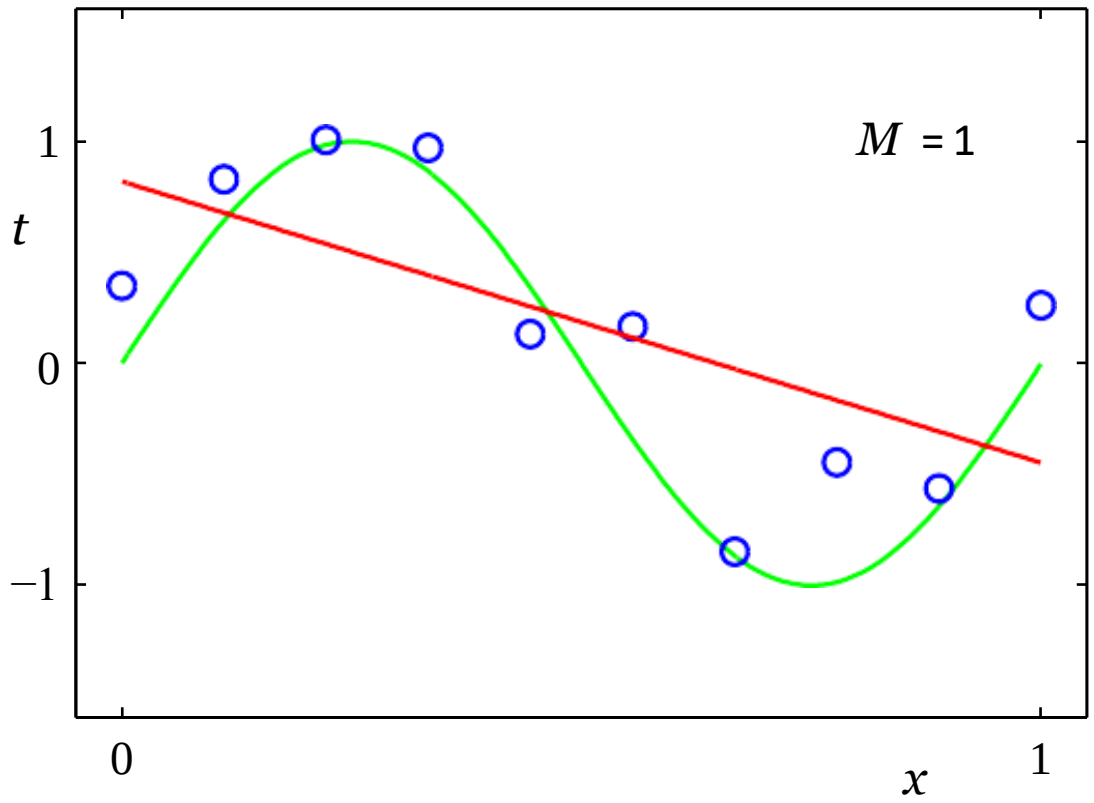
$$y = w_0$$



[Pattern Recognition and Machine Learning, Christopher Bishop.]

Polynomial Feature Mapping with $M = 1$

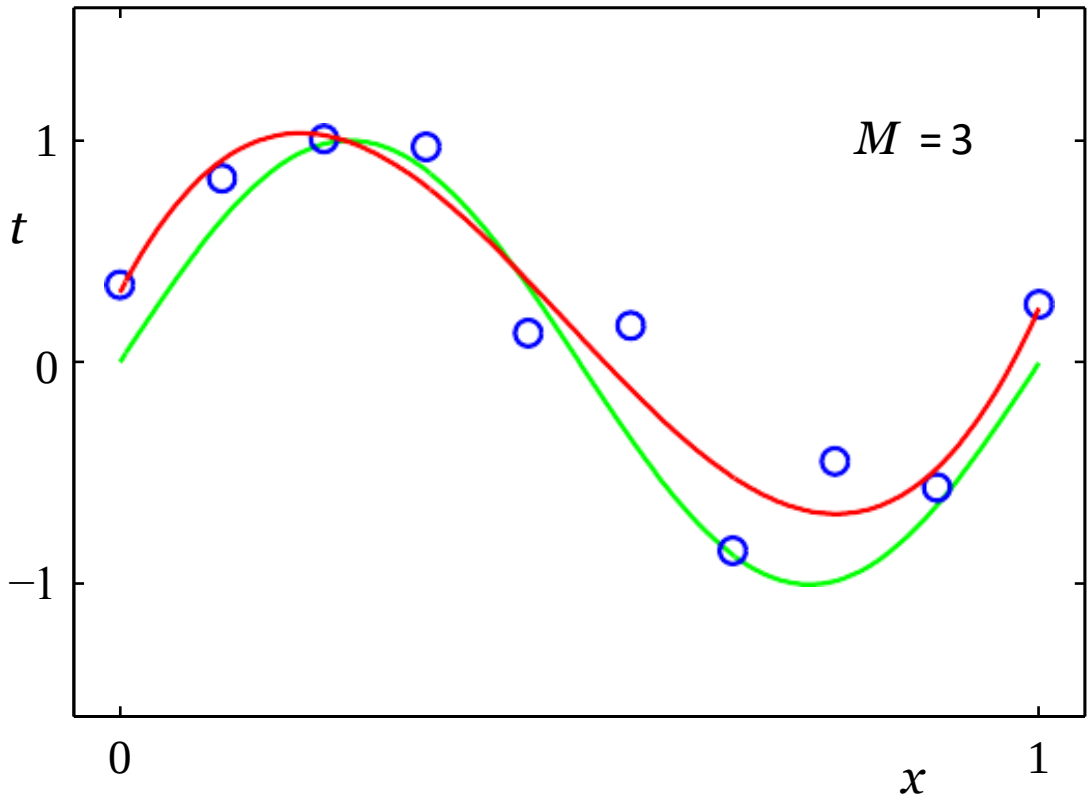
$$y = w_0 + w_1 x$$



[Pattern Recognition and Machine Learning, Christopher Bishop.]

Polynomial Feature Mapping with $M = 3$

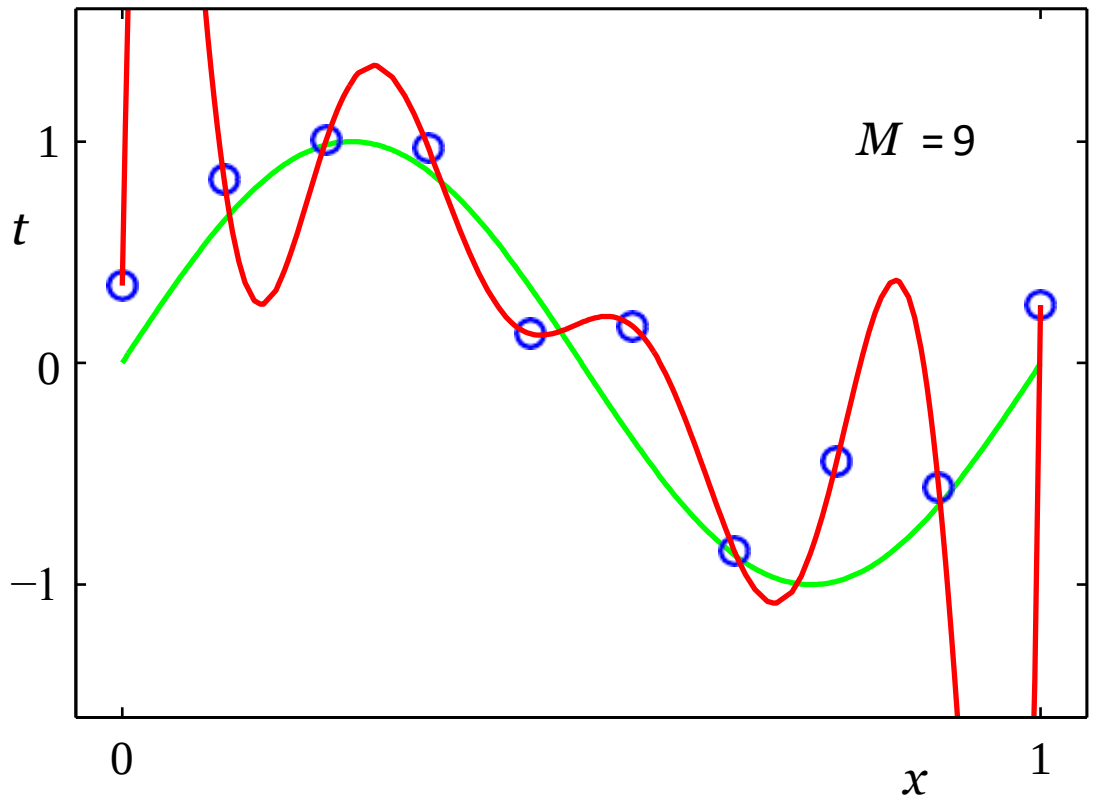
$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$$



[Pattern Recognition and Machine Learning, Christopher Bishop.]

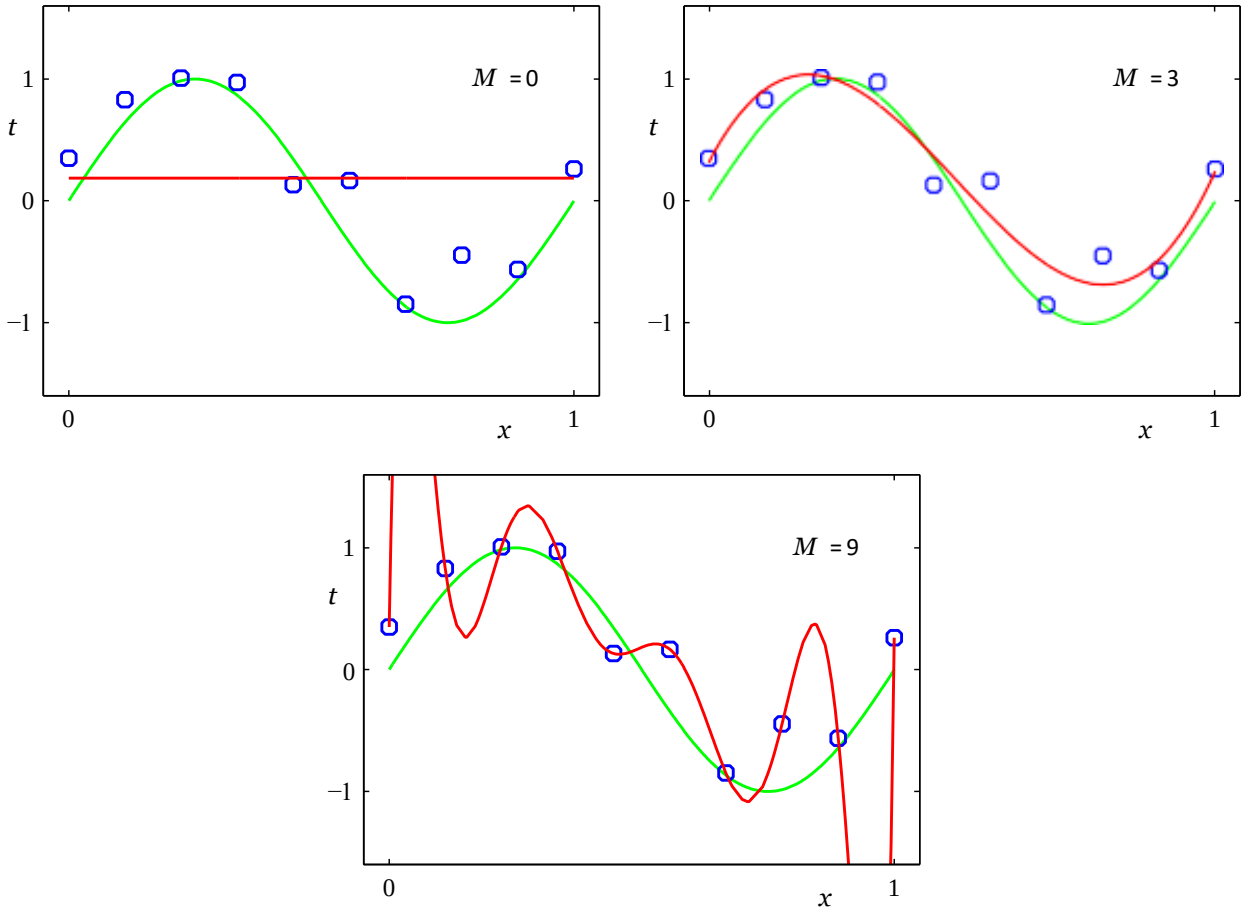
Polynomial Feature Mapping with $M = 9$

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + \ldots + w_9 x^9$$



[Pattern Recognition and Machine Learning, Christopher Bishop.]

Model Complexity and Generalization



Under-fitting (M=0):
Model is too simple, doesn't fit data well.

Good model (M=3):
Small test error, generalizes well.

Over-fitting (M=9):
Model is too complex, fits data perfectly.

Techniques for feature mapping

There are several techniques for feature mapping, including:

1.Feature extraction: This involves transforming the input data into a new set of features that capture the most important information. For example, in image processing, features such as edges, corners, and textures can be extracted from the image.

2.Feature transformation: This involves applying mathematical functions to the input data to transform it into a new set of features. For example, in text classification, the input text can be transformed into a bag-of-words representation, where each word in the text is represented by a count of its frequency.

3.Feature selection: This involves selecting a subset of the available features that are most relevant to the task at hand.

4.Feature scaling: This involves scaling the features to ensure that they have similar ranges and are on the same scale. This is important for some algorithms, such as those based on distance metrics.

Techniques for feature mapping

5. **Feature mapping** can be a time-consuming and iterative process, as different feature combinations and transformations may need to be tried to find the best set of features for a given task. However, effective feature mapping can significantly improve the performance of a machine learning algorithm and enable it to make more accurate predictions.

6.Feature engineering: This is the process of creating new features from the existing ones in order to improve the performance of a machine learning algorithm. Feature engineering can involve a combination of feature extraction, transformation, and selection techniques. For example, in a fraud detection task, a new feature could be created by calculating the ratio of the transaction amount to the average transaction amount for that user.

7.Dimensionality reduction: This involves reducing the number of features in a dataset while retaining the most important information. Dimensionality reduction can help to reduce overfitting and improve the computational efficiency of a machine learning algorithm.

Techniques for feature mapping

8. **Embeddings:** An embedding is a vector representation of a feature or object that captures its semantic meaning. For example, in natural language processing, word embeddings can be used to represent words as dense vectors in a high-dimensional space, where words with similar meanings are closer together. Embeddings can be learned using techniques such as word2vec or GloVe.

9.Data augmentation: This involves creating new examples by applying transformations to the existing data. For example, in image classification, data augmentation can involve rotating, flipping, or cropping the images to create new variations. Data augmentation can help to increase the size of the training set and improve the robustness of a machine learning model.

10.Domain knowledge: Expert knowledge about the domain in which the machine learning algorithm will be applied can be used to guide the feature mapping process. For example, in a medical diagnosis task, domain knowledge about the symptoms and risk factors of a particular disease can be used to select or engineer relevant features.

Thanks